GUIA PRÁCTICA – USO DE PROVEEDRO DE MBaaS

Autor: Ing. Edenilson López

OBJETIVOS:

Desarrollar una App que haga uso del MBaaS FireBase y sus servicios de base de datos NoSQL FireStore.

INTRODUCCIÓN:

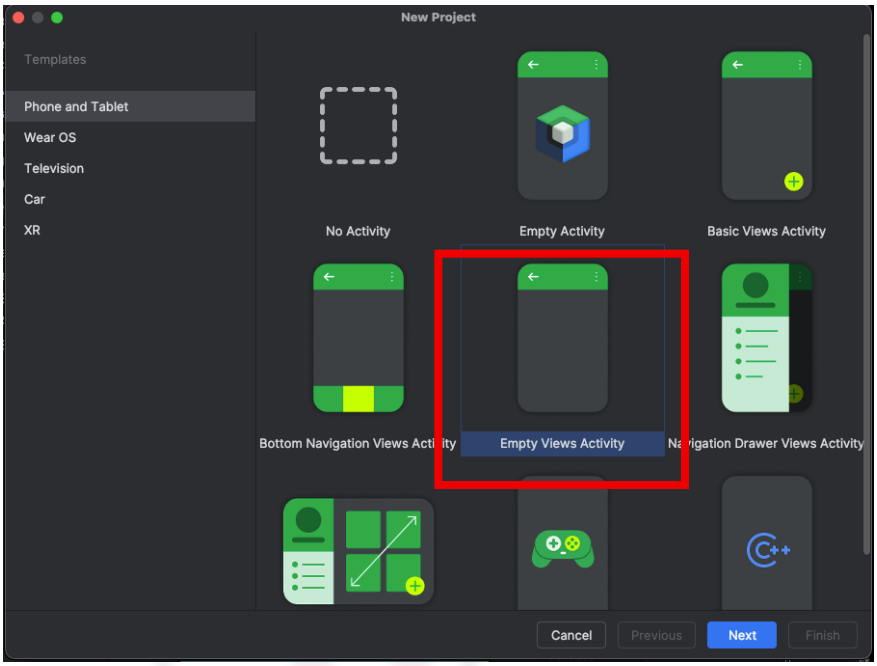
La presente guía desarrolla un ejercicio práctico que permite desarrollar una App que se conectará a una base de datos FireStore del proveedor de MBaaS Google Firebase para poder realizar el proceso de CRUD de los registros (documentos) de la base de datos.

DESARROLLO DE LA GUÍA PRÁCTICA

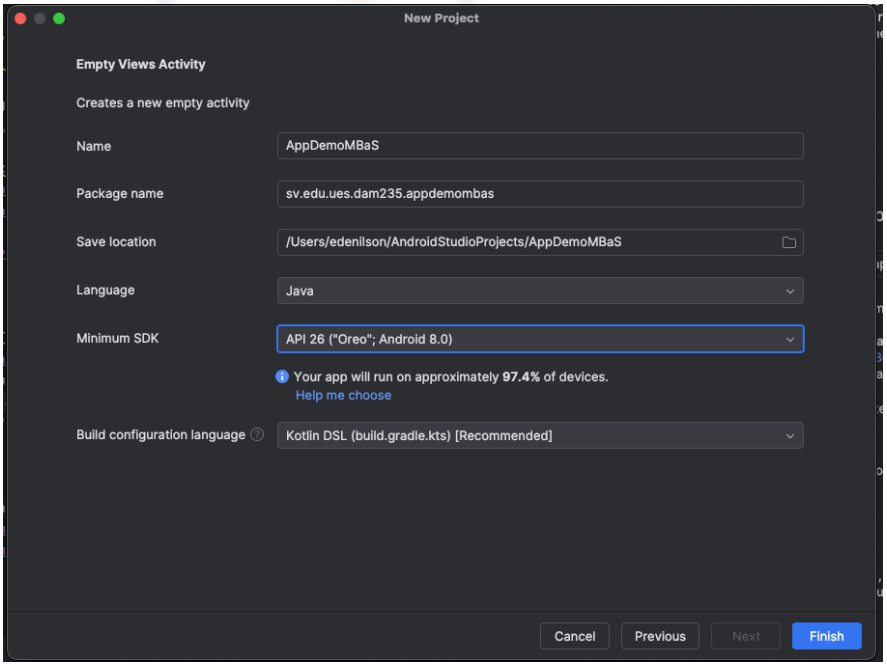
Contexto del Entorno de desarrollo:

La aplicación esta desarrollada en Android Studio Narwhal 3 Feature Drop, por lo que se recomienda realizar un upgrade del IDE y de todos sus componentes.

Paso 1: Crear un proyecto con una vista de actividad vacía.



Para el nombre de este proyecto usaremos **AppDemoMBaS**, lenguaje **Java** y **Min SDK 26**.



Paso 2. Modificar AndroidManifest para agregar los permisos y la metada necesaria para usar una FireBase, se agrega el permiso de uso de internet y el servicio GSM (Servicios Móviles de Google).

activity_main.xml | MainActivity.java | AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <queries>
        <package android:name="com.google.android.gms" />
    </queries>
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="AppDemoMBaS"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppDemoMBaS">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Paso 3. Agregar as dependencias de FireBase y Google GSM.

En el archivo de modulo **build.gradle.kts,** agregar la línea siguiente y sincronizar el proyecto.

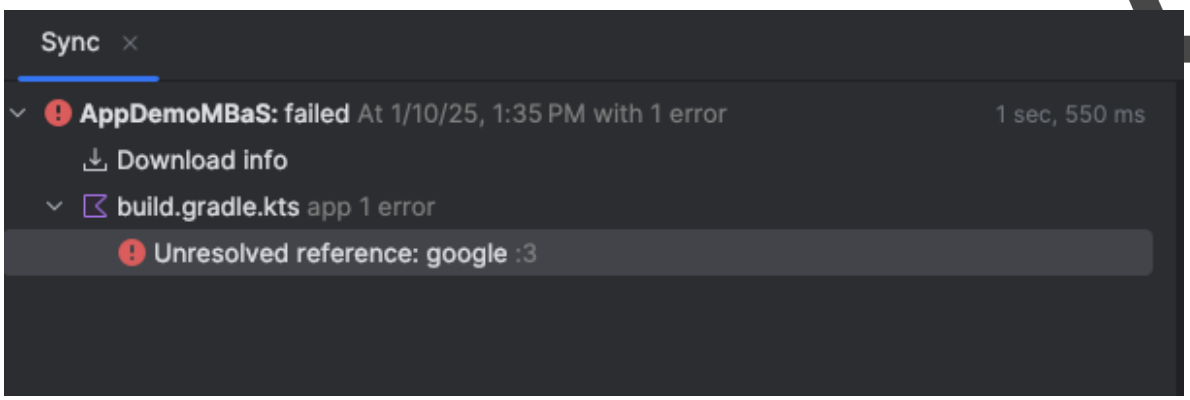
```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.google.gms.google.services)
}
```

```
dependencies {

    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.ext.junit)
    androidTestImplementation(libs.espresso.core)
    implementation(libs.firebase.firestore)
    implementation(platform( notation = "com.google.firebase:firebase-bom:34.2.0"))
    implementation("com.google.firebase:firebase-analytics")
}
```



Si al sincronizar obtiene un error como el siguiente



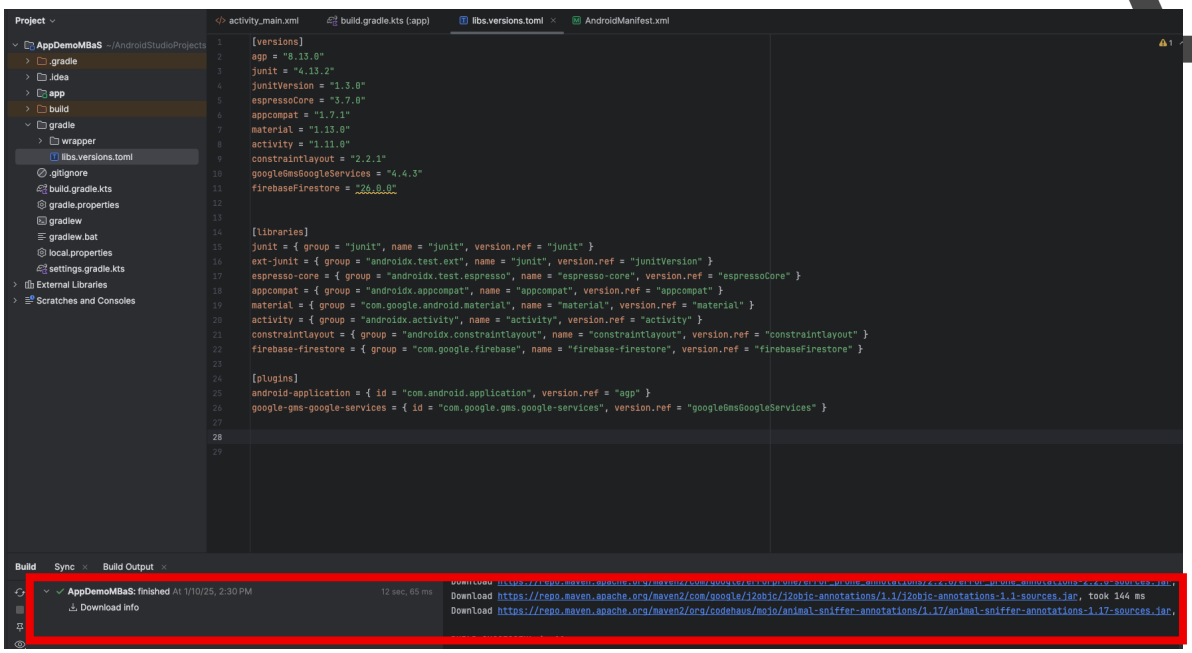
Es por que no se han incluido adecuadamente las dependencias de Firestore y de Google GSM. Procederemos de la siguiente manera.

Cambiaremos a la visita de modo Proyecto y editaremos el archivo libs.versions.toml de la carpeta gradle.

Agregaremos las líneas que se marcan sobre la imagen siguiente:



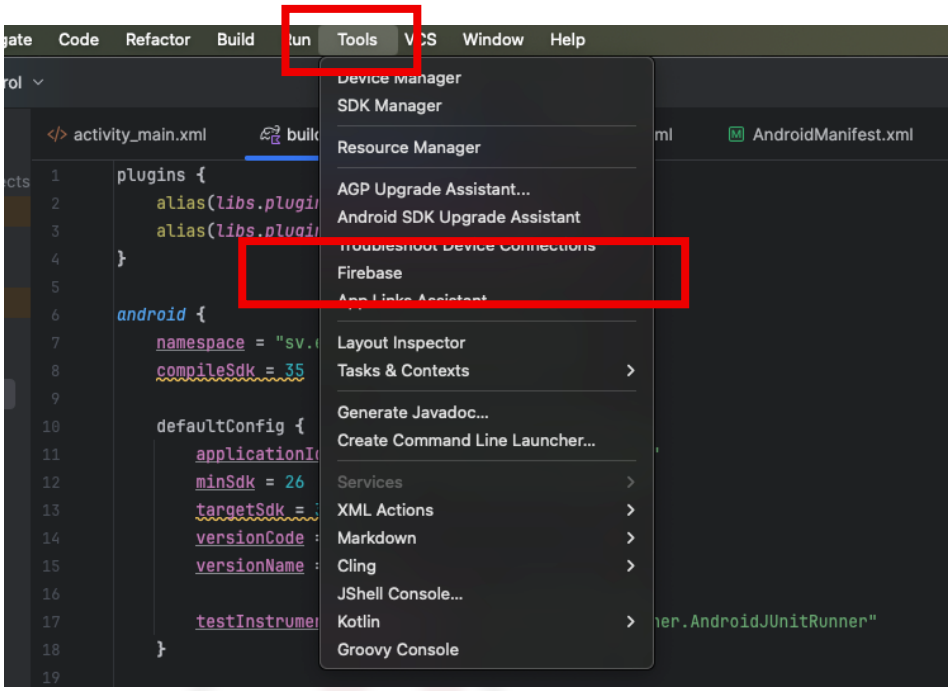
Y luego sincronizamos el proyecto nuevamente.



El proyecto se sincronizará correctamente.

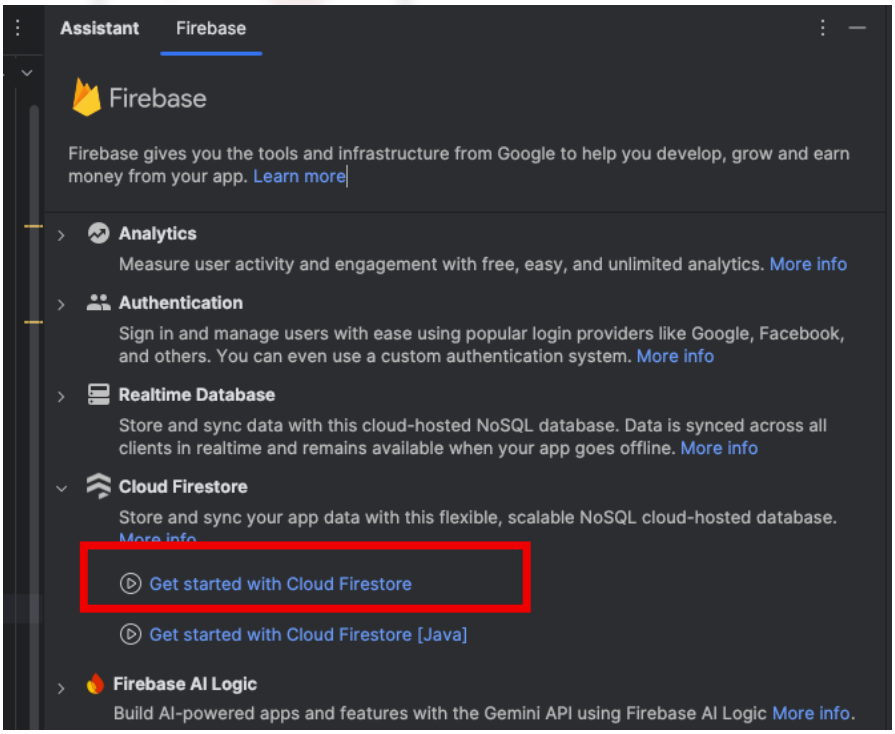
Paso 4. Conectar el proyecto de la App con FireBase.

En la barra de menú de inde seleccionaremos Tools -> Firebase

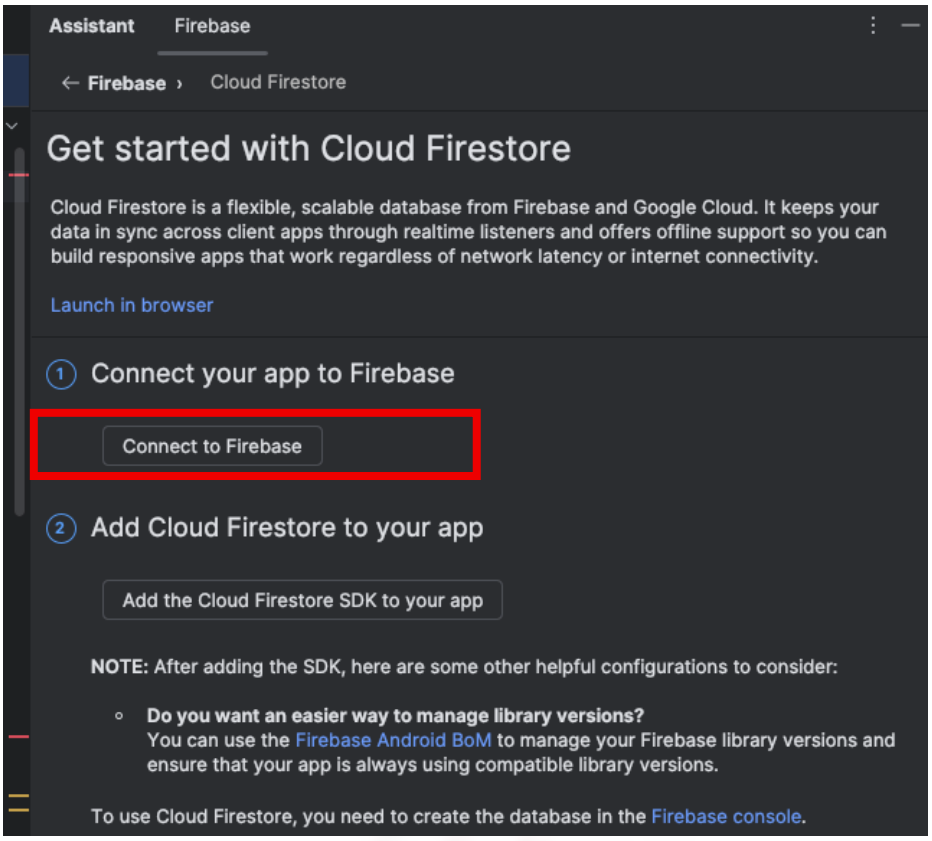


Se mostrará el panel del asistente de Firebase para la configuración del proyecto.

Buscaremos la opción Cloud Firestore y seleccionaremos la opción **Get starter with Cloud Firestore**



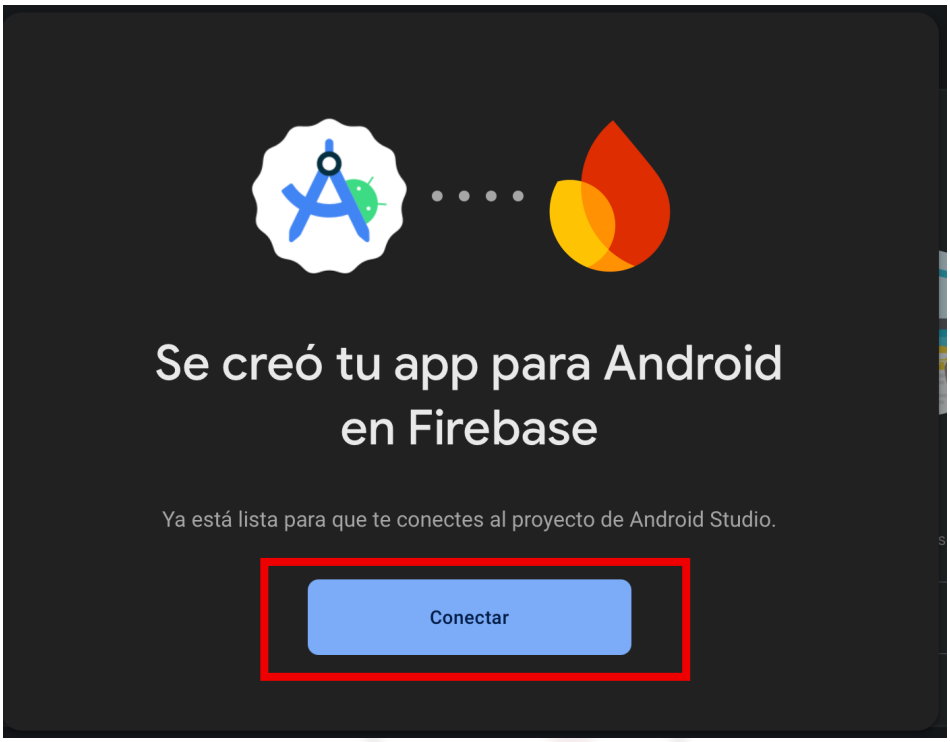
Se nos mostrará una ventana del asistente que nos permitirá conectarnos con el proyecto credo en Firebase en la guía anteriormente desarrollada. Seleccionaremos la opción **Connect to Firebase**.



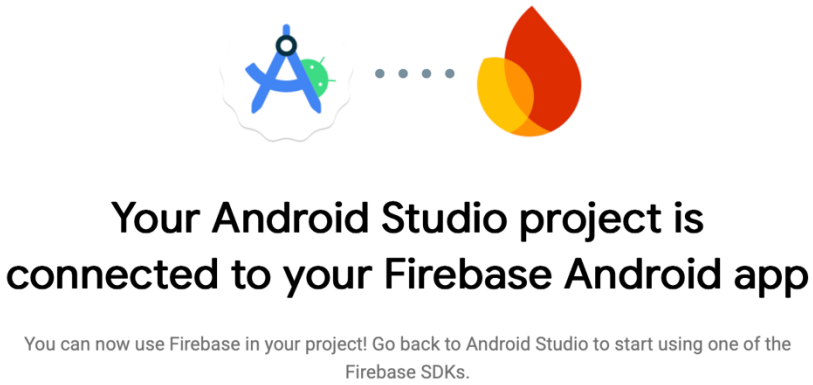
Debemos de tener previamente iniciada la sesión en la consola de Firebase para que se nos abra una ventana del navegador directamente conectado al sitio de Firebase y mostrándonos los proyectos que tengamos creados para seleccionar a cuál nos conectaremos desde el proyecto de Android.



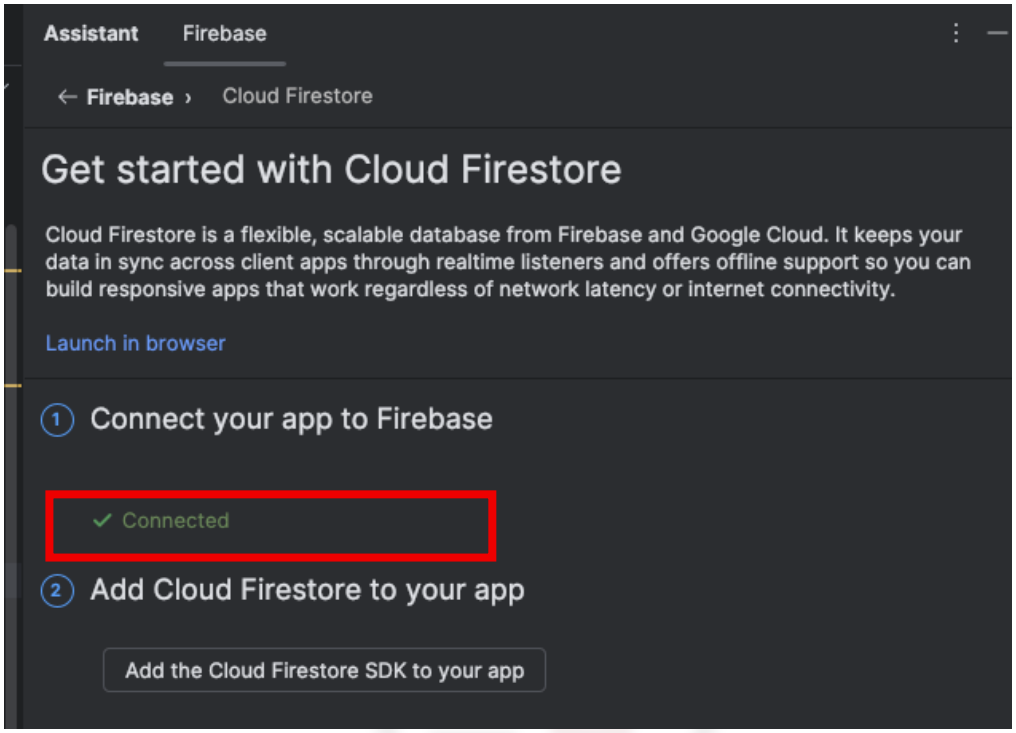
Seleccionamos el proyecto dando un clic y nos mostrara una ventana como la siguiente:



Al dar un clic en el botón conectar el asistente nos mostrara una ventana que indica el proceso de conexión entre la App y Firebase a finalizado.



Y en el asistente de Firebase se mostrará un mensaje que indica que el proyecto está conectado.



Paso 5. Crear la UI para la vista activity_main.xml.

Reemplazaremos la vista completa.

El código original de activity_main.xml es:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

Y se reemplazara por esto:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/buttonCargar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="48dp"
        android:layout_marginTop="272dp"
        android:text="Leer Colección de FireStore"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <ProgressBar
        android:id="@+id/progressBar"
        style="?android:attr/progressBarStyle"
        android:layout_width="106dp"
        android:layout_height="84dp"
        android:layout_marginStart="120dp"
        android:layout_marginTop="120dp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonCargar"
        tools:visibility="visible" />


    <TextView
        android:id="@+id/tvDocuments"
        android:layout_width="321dp"
        android:layout_height="345dp"
        android:layout_marginStart="48dp"
        android:layout_marginTop="24dp"
        android:textAlignment="gravity"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonCargar" />

    <EditText
        android:id="@+id/edtNombres"
        android:layout_width="258dp"
        android:layout_height="46dp"
        android:layout_marginStart="41dp"
        android:layout_marginTop="37dp"
        android:ems="10"
        android:hint="Ingrese nombres..."
```

```
        android:inputType="text"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <EditText
        android:id="@+id/edtApellidos"
        android:layout_width="262dp"
        android:layout_height="55dp"
        android:layout_marginStart="41dp"
        android:layout_marginTop="15dp"
        android:ems="10"
        android:hint="Ingrese apellidos..."
        android:inputType="text"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/edtNombres" />

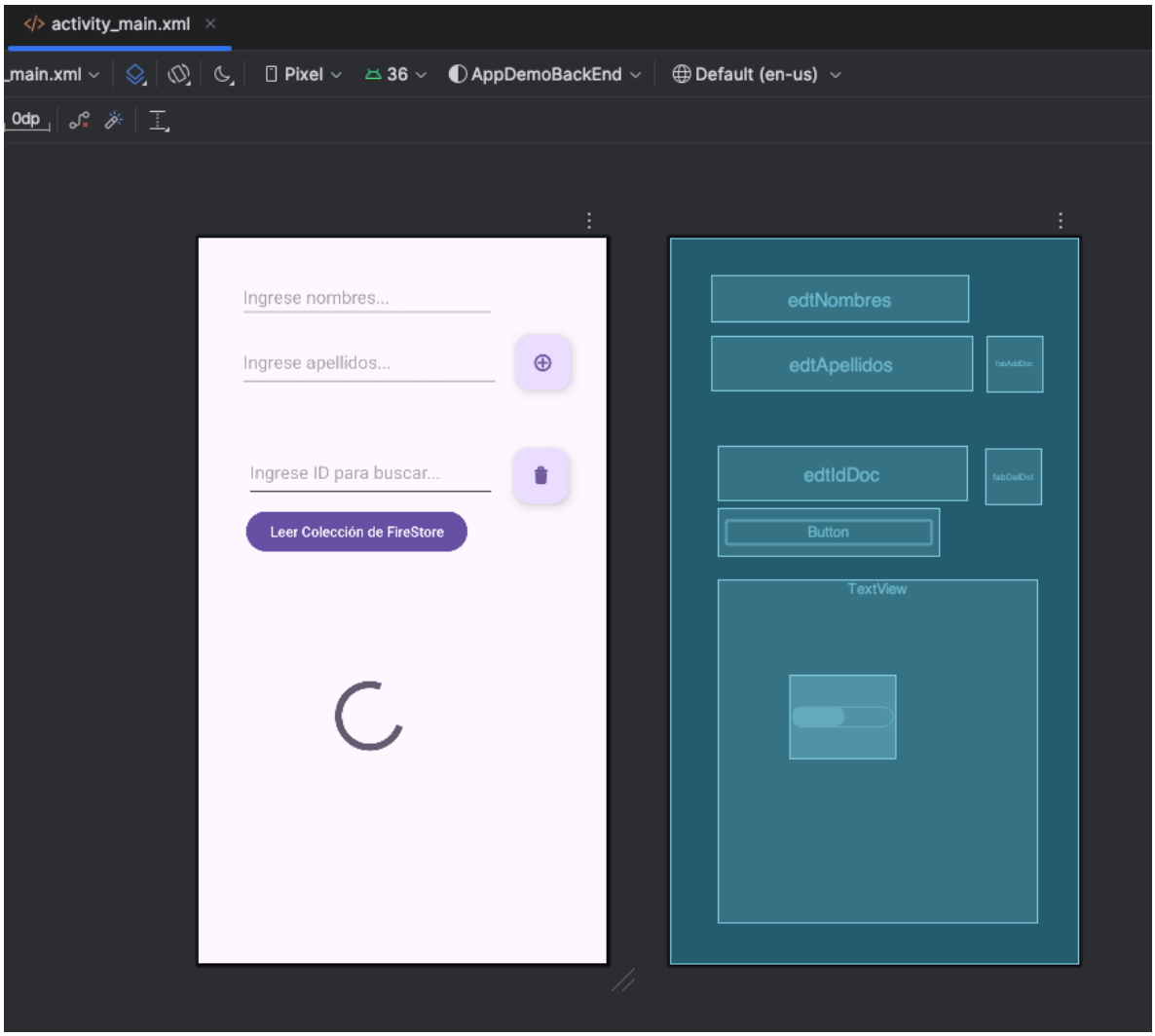

<com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabAddDoc"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="98dp"
        android:layout_marginEnd="36dp"
        android:clickable="true"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/edtApellidos"
        app:layout_constraintTop_toTopOf="parent"
        app:srcCompat="@android:drawable/ic_menu_add" />

    <EditText
        android:id="@+id/edtIdDoc"
        android:layout_width="251dp"
        android:layout_height="55dp"
        android:layout_marginStart="48dp"
        android:layout_marginBottom="8dp"
        android:ems="10"
        android:hint="Ingrese ID para buscar..."
        android:inputType="text"
        app:layout_constraintBottom_toTopOf="@+id/buttonCargar"
        app:layout_constraintStart_toStartOf="parent" />


<com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabDelDol"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="18dp"
        android:layout_marginTop="58dp"
        android:layout_marginEnd="38dp"
        android:clickable="true"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/edtIdDoc"
        app:layout_constraintTop_toBottomOf="@+id/fabAddDoc"
        app:srcCompat="@android:drawable/ic_menu_delete" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

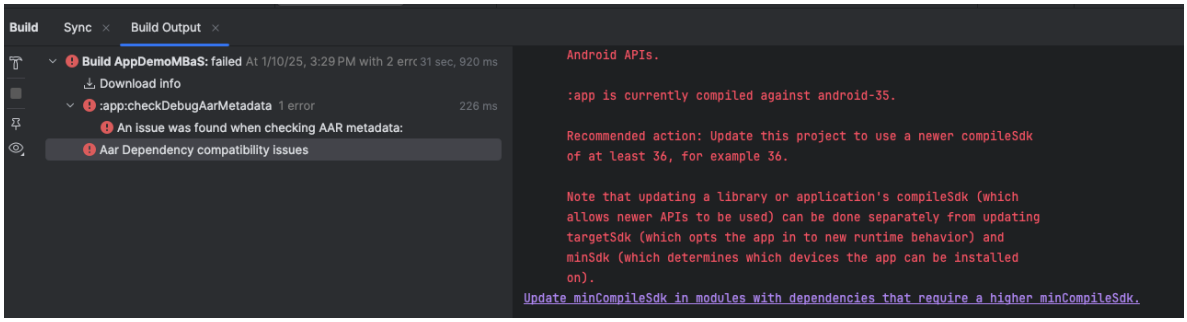
La vista de diseño mostrara lo siguiente UI.



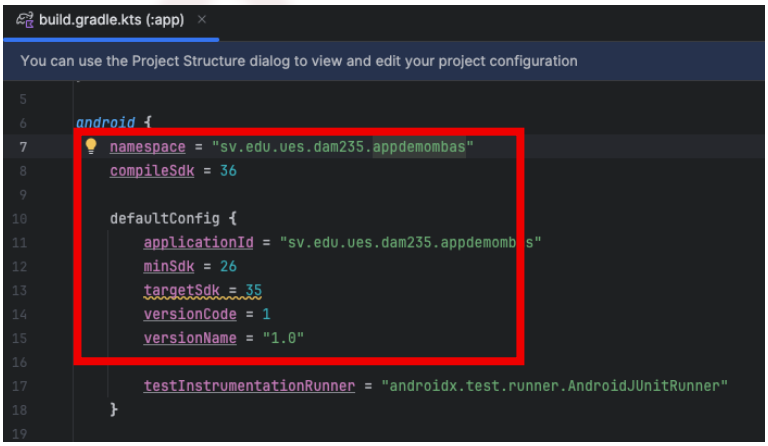
Paso 5. Ejecutamos la aplicación para probar su funcionamiento.

Nota: El emulador usado para ejecutar este App es Pixel 8 Pro API 34

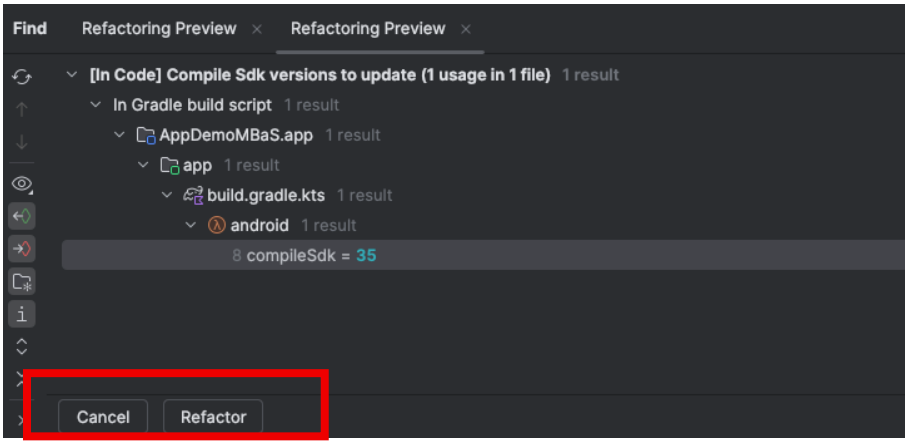
Si al ejecutar la App se nos muestra un error de minCompileSdk.



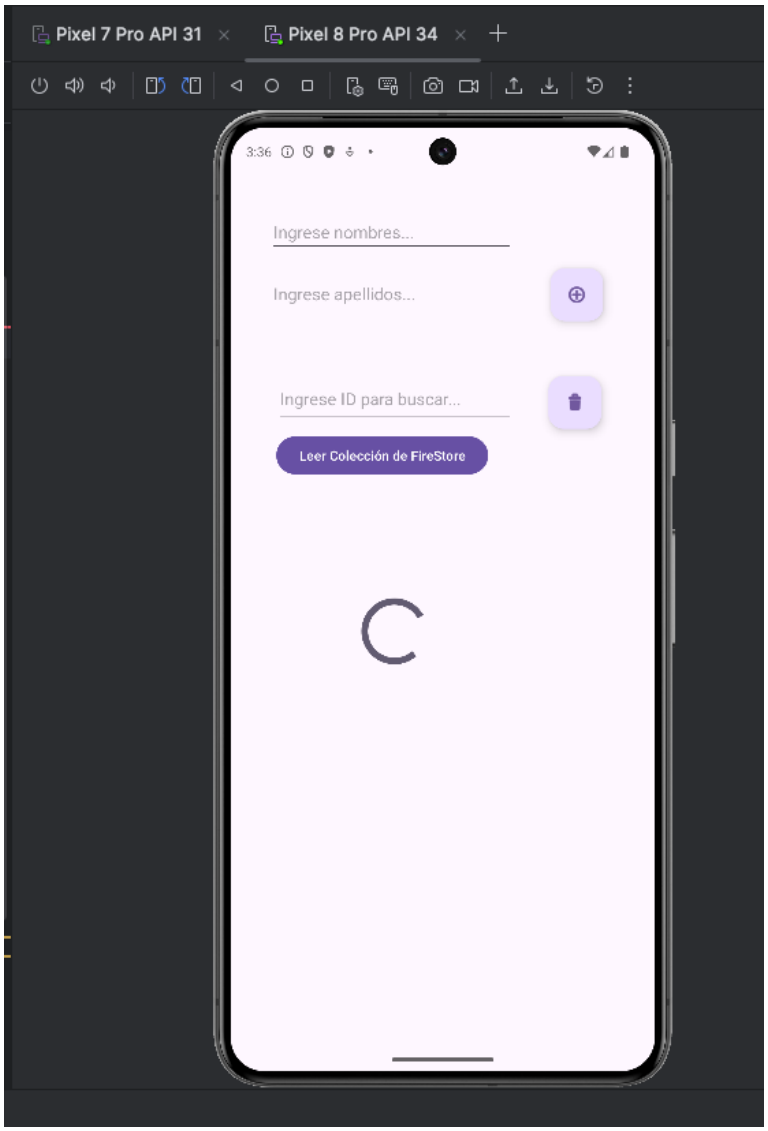
Realizaremos la refactorización del código para soportar la compilación de API 35 actualizando el archivo builds.gradle.kts (modulo)



Y luego refactorizamos.



Ejecutamos nuevamente y se mostrará la App en ejecución mostrando la UI siguiente:



PASO 6. Programación de MainActivity.java

Crearemos las siguientes propiedades de la clase principal MainActivity.java

```
public class MainActivity extends AppCompatActivity {

    Button btCargar;
    TextView tvJSON;
    ProgressBar progressBar;
    FloatingActionButton fabAddDoc, fabDelDoc;
    EditText edtNombres, edtApellidos,edtID;
    FirebaseFirestore db;
```

Y luego las inicializamos en el método onCreate()

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
(v, insets) -> {
        Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
        return insets;

    });

    db = FirebaseFirestore.getInstance();

    btCargar = findViewById(R.id.buttonCargar);
    tvJSON  = findViewById(R.id.tvDocuments);
    progressBar = findViewById(R.id.progressBar);
    progressBar.setVisibility(View.GONE);
    edtNombres = findViewById(R.id.edtNombres);
    edtApellidos = findViewById(R.id.edtApellidos);
    edtApellidos = findViewById(R.id.edtApellidos);
    edtID = findViewById(R.id.edtIdDoc);
    fabAddDoc = findViewById(R.id.fabAddDoc);
    fabDelDoc = findViewById(R.id.fabDelDol);
```

Programaremos el código necesario para realizar la consulta de los documentos existentes en la base de datos Firestore. Esto lo realizaremos programando el método onClick() del fabbutton btCargar y creando un método personalizado llamado actualizarListaDocumentos(). El código para agregar es el siguiente:

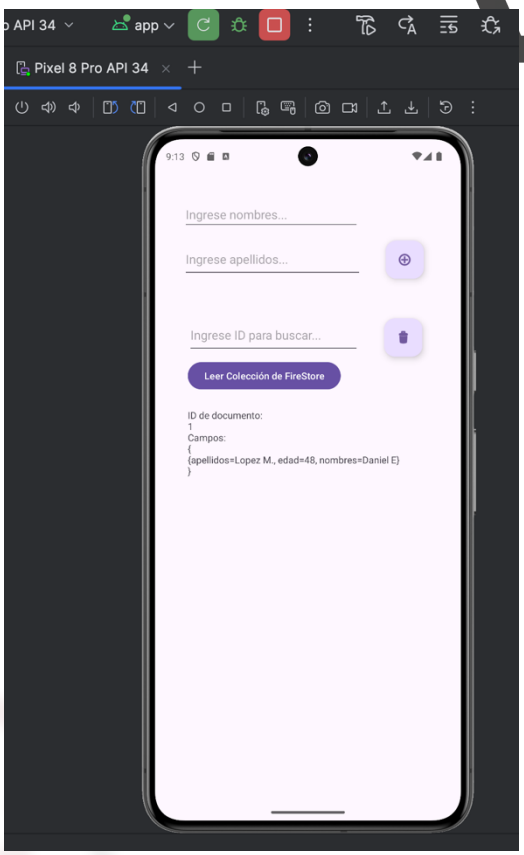
```
btCargar.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        actualizarListaDocumentos();
    }
});
```

```
private void actualizarListaDocumentos() {
    tvJSON.setText("");
    progressBar.setVisibility(View.VISIBLE);
    StringBuilder documento = new StringBuilder();
    db.collection("alumnos")
            .get()
            .addOnCompleteListener(new
OnCompleteListener<QuerySnapshot>() {
                @Override
                public void onComplete(@NonNull Task<QuerySnapshot> task)
{
                    progressBar.setVisibility(View.GONE);
                    if (task.isSuccessful()) {
                        if (task.getResult().size() == 0) {
                            tvJSON.setText("No hay documentos que
mostrar...")

                        } else {
                            for (QueryDocumentSnapshot document :
task.getResult()) {
                                documento.append("ID de documento: \n")
                                        .append(document.getId())
                                        .append("\nCampos:")
                                        .append("\n{")

.append("\n").append(document.getData())
                                        .append("\n}\n\n");
                                tvJSON.setText(documento);
                            }
                        }
                    } else {
                        tvJSON.setText("Error al obtener documentos.");
                    }
                }
            });
}
```

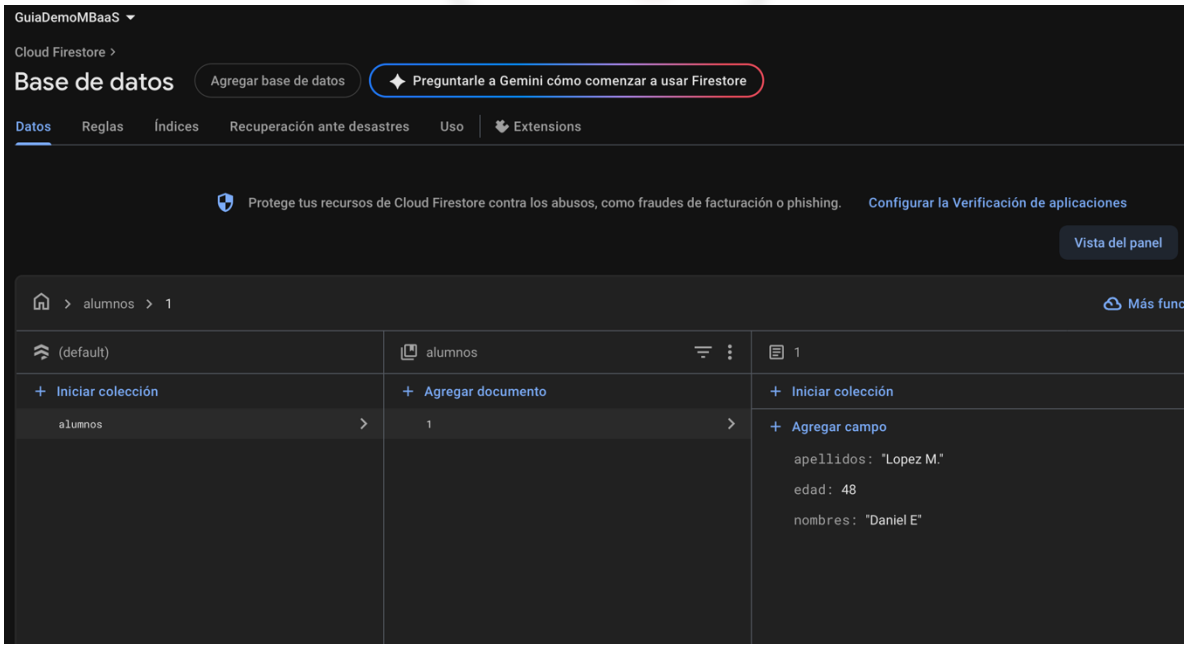
Probaremos la aplicación para ver el resultado del botón Cargar.



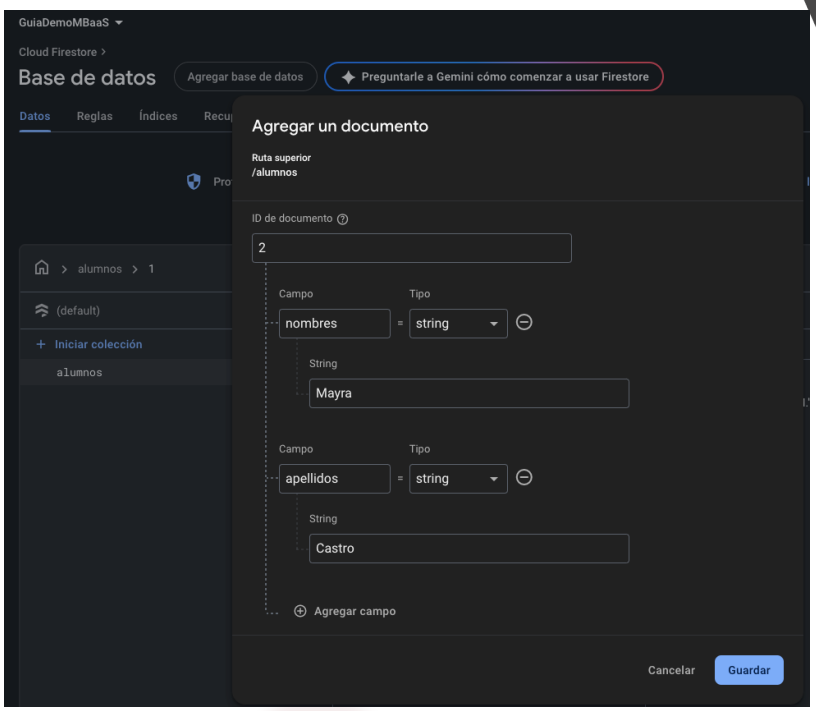
Al dar un clic en el botón **"Leer Colección de FireStore"** obtendremos una cadena de texto que fue formateado de tal manera que simbolice una estructura de datos tipo json.

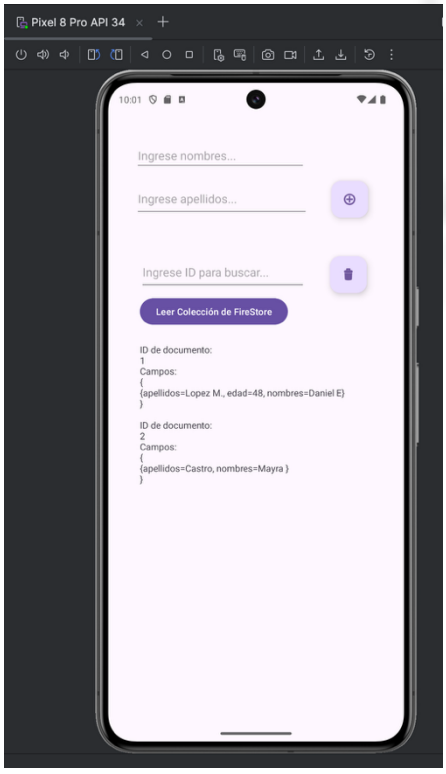
Esta cadena corresponde al único documento de la base datos.

Con este código habremos creado nuestra primera consulta a la base de datos Firestore en la colección **alumnos**



Agregaremos un registro manual mente y probaremos nuevamente.





Al ejecutar a aplicación se puede apreciar que se están leyendo dos documentos de la base de datos.

Continuaremos programando la operación de agregar un documento desde el fab button de agregar registro.

Para agregar un nuevo registro a la base de datos de Firestore podemos hacerlo de dos maneras, una es usando una clase Map y la otra usando una clase POJO (Plain Old Java Object), para esta guía usaremos una clase POJO de nombre Alumno. El código para construir la clase Alumno.java, con sus propiedades, constructores, métodos, gatter y setter es el siguiente:

```
public class Alumno {
    public Alumno(String nombres, String apellidos) {
        this.nombres = nombres;
        this.apellidos = apellidos;

    }
    public String nombres;
    public String apellidos;

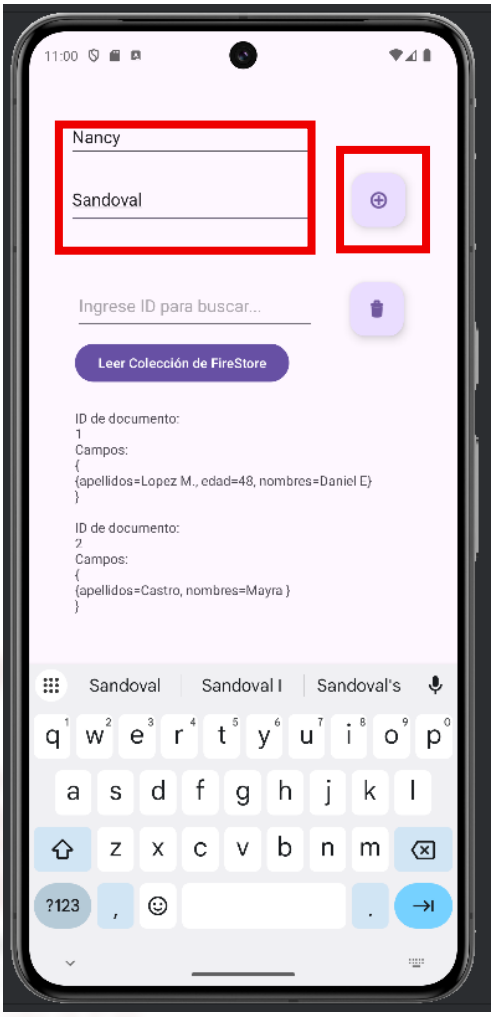
    public Map<String, Object> toMap() {
        Map<String, Object> result = new HashMap<>();
        result.put("nombres", nombres);
        result.put("apellidos", apellidos);
        return result;
    }

    public static Alumno fromMap(Map<String, Object> map) {
        String nombres = (String) map.get("nombres");
        String apellidos = (String) map.get("apellidos");
        return new Alumno(nombres, apellidos);
    }

    @Override
    public String toString() {
        return "Alumno{" +
                "nombres='" + nombres + '\'' +
                ", apellidos='" + apellidos + '\'' +
                '}';
    }
    //Getters y Setters
    public String getNombres() {
        return nombres;
    }
    public String getApellidos() {
        return apellidos;
    }
    public void setNombres(String nombres) {
        this.nombres = nombres;
    }
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }
}
```

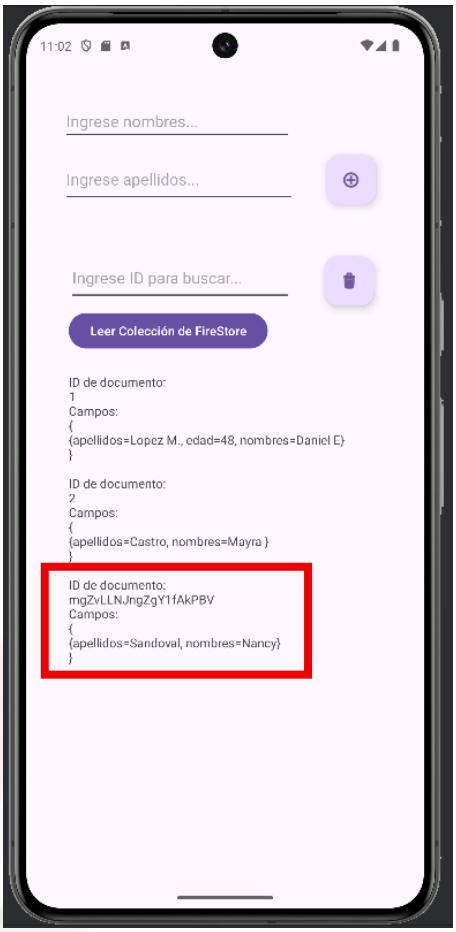
Ahora agregaremos el método onClick del fab button fabAddDoc. El código es el siguiente:

```
fabAddDoc.setOnClickListener(new View.OnClickListener(){
    @Override
    public void onClick(View view) {
        String nombres = edtNombres.getText().toString();
        String apellidos = edtApellidos.getText().toString();
        if (!nombres.isEmpty() && !apellidos.isEmpty()) {
            db.collection("alumnos").add(new Alumno(nombres,
apellidos).toMap());
            Toast.makeText(MainActivity.this, "Documento creado",
Toast.LENGTH_SHORT).show();
           actualizarListaDocumentos(view);
            edtNombres.setText("");
            edtApellidos.setText("");
        } else {
            Toast.makeText(MainActivity.this, "Ingrese nombres y
apellidos", Toast.LENGTH_SHORT).show();
        }
    }
});
```



En el método se obtienen los valores que se hayan ingresado para nombres y apellidos y luego mediante el método **add()** de la clase **collection** de Firestore y usando una clase Alumno se crea un nuevo documento con el id aleatorio. Ejecutamos la aplicación y verificamos su funcionamiento.

Luego de llenar los campos y presionar el botón Add, se mostrará un mensaje indicando que un documento se ha agregado, se invocará el método **actualizarListadoDocumentos()** y en el textview se mostrará la información del nuevo registro agregado.



El siguiente paso será programar el fab button eliminar registro.

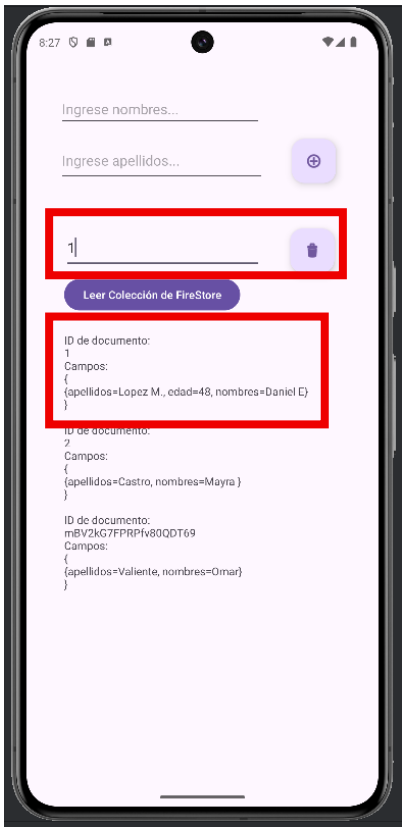


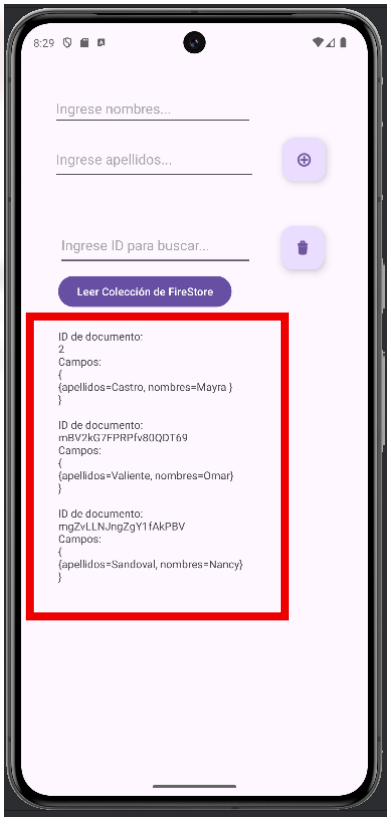
Para esto agregaremos el siguiente código.

```
fabDelDoc.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        String idDoc = edtID.getText().toString();
        if (!idDoc.isEmpty()) {
            db.collection("alumnos").document(idDoc).delete();
            Toast.makeText(MainActivity.this, "Documento eliminado",
Toast.LENGTH_SHORT).show();
            edtID.setText("");
            actualizarListaDocumentos(view);
        } else {
            Toast.makeText(MainActivity.this, "Ingrese el ID del
documento a eliminar", Toast.LENGTH_SHORT).show();
        }
    }
});
```

El código anterior obtiene el valor del cuadro de texto edtID el cual será usado para ingresar el ID del registro a eliminar, dicho ID será pasado como parámetro a la clase **document** y sobre el cual se ejecutará el método **delete()**. SI el documento existe se eliminará y se mostrara un mensaje de "Documento eliminado", en caso contrario se mostrará el mensaje indicando que se "Ingrese el ID del documento a eliminar".

Ejecutáremos la App utilizando el ID 1, luego de dar clic en el fab button eliminar se mostrará el mensaje de documento eliminado, se limpiara el edtID y se actualizará la lista





Finalizada la programación de la lógica de eliminación de documentos vamos a programar la lógica de actualización.

Para esto formularemos la siguiente lógica:

Si el edtID tiene un ID valido y los campos nombre y apellidos son distintos cadena vacía, se realizará la actualización del documento con ese ID cuando se dé un clic en el fad button fabAddDoc y se mostrará un mensaje de documento actualizado.

De lo contrario, es decir si el ID no sea válido (no se encuentra en la base de datos) se agregará un nuevo registro a la base de datos.

Reemplazaremos el código del evento onClick de fab button fabAddDoc con el código siguiente:

```
fabAddDoc.setOnClickListener(new View.OnClickListener(){
    @Override
    public void onClick(View view) {

        if(!edtApellidos.getText().toString().isEmpty() &&
!edtNombres.getText().toString().isEmpty() &&
!edtID.getText().toString().isEmpty()){
            actualizarDocumento(edtID.getText().toString());
            return;
        }

        String nombres = edtNombres.getText().toString();
        String apellidos = edtApellidos.getText().toString();
        if (!nombres.isEmpty() && !apellidos.isEmpty()) {
            db.collection("alumnos").add(new Alumno(nombres,
apellidos).toMap());
            Toast.makeText(MainActivity.this, "Documento creado",
Toast.LENGTH_SHORT).show();
            actualizarListaDocumentos();
            edtNombres.setText("");
            edtApellidos.setText("");
        } else {
            Toast.makeText(MainActivity.this, "Ingrese nombres y
apellidos", Toast.LENGTH_SHORT).show();
        }
    }
});
```

Y crearemos el método actualizarDocumento() con el código siguiente:

```
private void actualizarDocumento(String idDoc) {


db.collection("alumnos").document(idDoc).get().addOnCompleteListener(new
OnCompleteListener<DocumentSnapshot>() {
        @Override
        public void onComplete(@NonNull Task<DocumentSnapshot> task) {
            if (task.isSuccessful()) {
                DocumentSnapshot document = task.getResult();
                if (document.exists()) {

                    if (document.getData() != null) {

db.collection("alumnos").document(idDoc).update("nombres",
edtNombres.getText().toString(),
                                    "apellidos",
edtApellidos.getText().toString());
                        Toast.makeText(MainActivity.this, "Documento
actualizado", Toast.LENGTH_SHORT).show();
                        actualizarLIstaDocumentos();
                        edtNombres.setText("");
                        edtApellidos.setText("");
                        edtID.setText("");
                    }
                } else {
                    Toast.makeText(MainActivity.this, "No existe el
documento a actualizar", Toast.LENGTH_SHORT).show();
                }
            }
        }

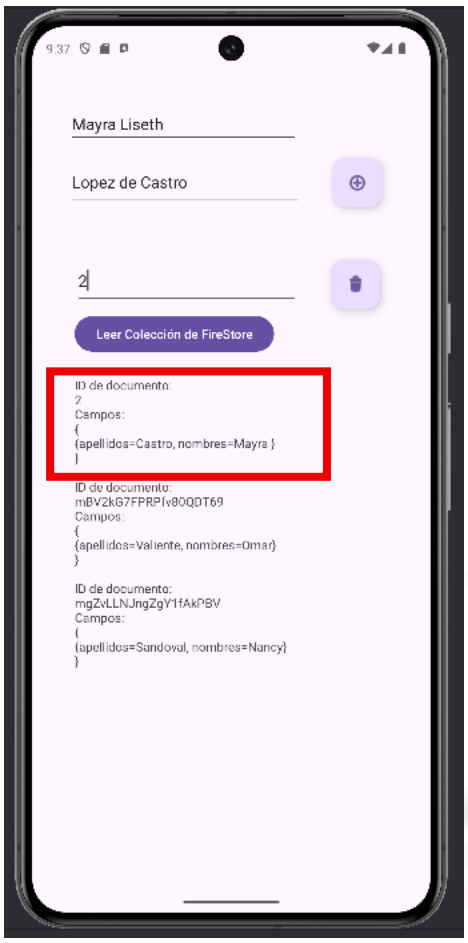
    });

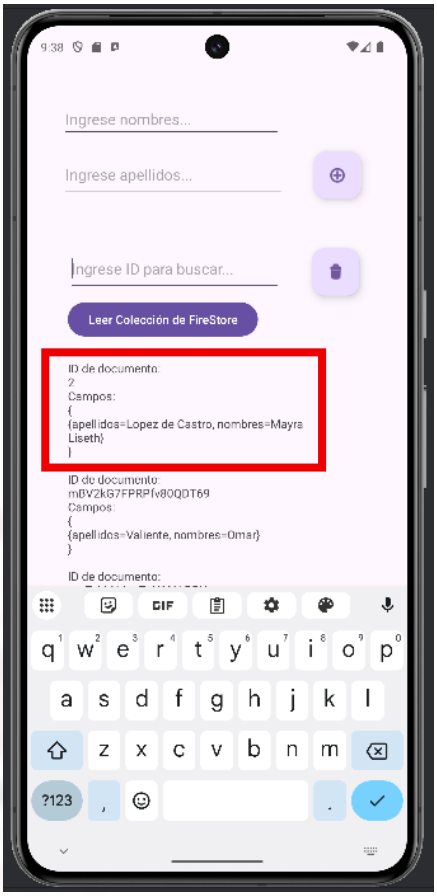
}
```

El código anterior usa el método **update()** con los parámetros a actualizar sobre el elemento documento que es devuelto por:

db.collection("alumnos").document(idDoc)

A continuación, realizaremos la prueba de actualización de datos sobre el documento con ID 2.





Luego de haber dado un clic en el fab button fabAddDoc estado los campos edtIdDoc, edtNombres y edtApellidos, el programa actualizara los valores de las propiedades del documento para el ID 2, luego actualizara la lista de elementos de la base de datos y limpiara los elementos EditText.

Finalizaremos esta guía con una actualización al evento onClick del botón buttonCargar.

Modificaremos el evento onClick de dicho botón para que si existe una cadena de texto en el campo edtIdDoc cuando se presione ese botón se realice la busque de ese ID de documento, en caso no que es campo este vacío pues seguirá devolviendo el listado completo de documentos de la base de datos y su colección "alumnos". El evento onClick del botón buttonCargar quedara con las siguientes líneas de código.

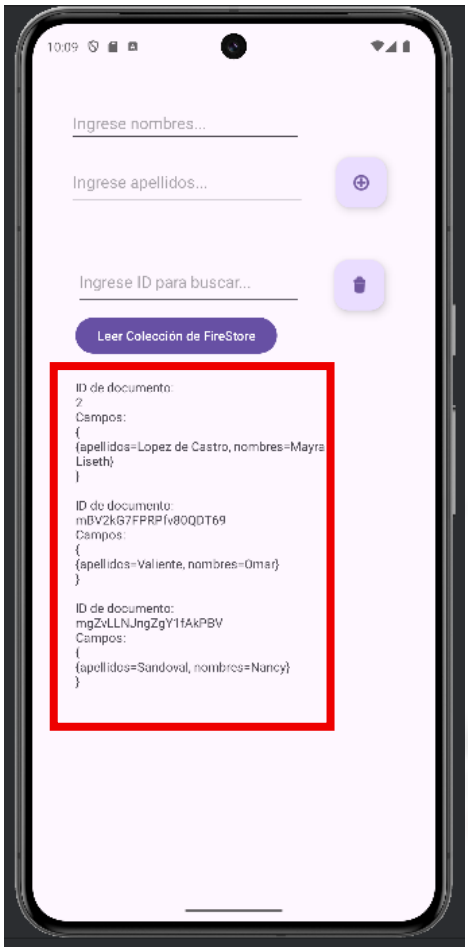
```
btCargar.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View view) {
        if(edtID.getText().toString().isEmpty()){
            actualizarListaDocumentos(view);
        }else{

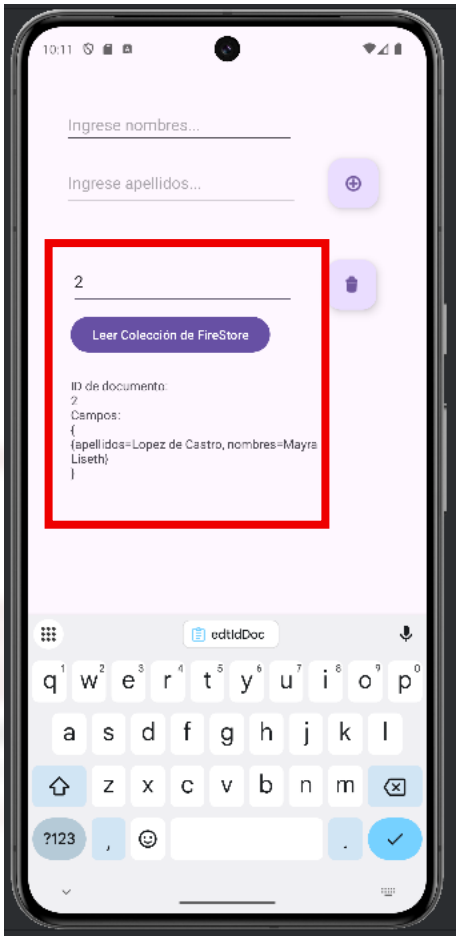
            String idDoc = edtID.getText().toString();

db.collection("alumnos").document(idDoc).get().addOnCompleteListener(new
OnCompleteListener<DocumentSnapshot>() {
                @Override
                public void onComplete(@NonNull Task<DocumentSnapshot>
task) {
                    if(task.isSuccessful()){
                        DocumentSnapshot document = task.getResult();
                        if(document.exists()){
                            StringBuilder documento = new
StringBuilder();
                            if(document.getData()!=null){
                                documento.append("ID de documento: \n")
                                        .append(document.getId())
                                        .append("\nCampos:")
                                        .append("\n{")

.append("\n").append(document.getData())
                                        .append("\n}\n\n");
                                tvJSON.setText(documento);
                            }
                        }else{
                            Toast.makeText(MainActivity.this, "No existe
el documento", Toast.LENGTH_SHORT).show();
                        }
                    }else{
                        Toast.makeText(MainActivity.this, "Error al
obtener el documento", Toast.LENGTH_SHORT).show();
                    }
                }
            });
        }
    }
});
```

Probaremos una vez más la aplicación en esta oportunidad nos evaluaremos que la búsqueda se realice por todos los documentos o por uno en particular llenando con un ID de documento el elemento edtIdDoc.





Comprobamos que la búsqueda por ID funciona correctamente.

Con eso eso concluido la guía de uso de MBaaS, en la cual hemos implementado las operaciones CRUD de una base de dato Firestore de Google Firebase.

Referencias.

- Google (s.f). Documentación de Firestore . *Obtén datos con Cloud Firestore*. Recuperado de https://firebase.google.com/docs/firestore/query-data/get-data?hl=es-419#java

- Google (s.f). Documentación de Firestore. *Borrar datos de Cloud Firestore*. Recuperado de https://firebase.google.com/docs/firestore/manage-data/delete-data?hl=es-419

Anexos.

Código de AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <queries>
        <package android:name="com.google.android.gms" />
    </queries>
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppDemoMBaS"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER"
/>
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Código de build.gradle.kts (modulo)

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.google.gms.google.services)
}

android {
    namespace = "sv.edu.ues.dam235.appdemombas"
    compileSdk = 36

    defaultConfig {
        applicationId = "sv.edu.ues.dam235.appdemombas"
        minSdk = 26
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner =
"androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {

    implementation(libs.appcompat)
    implementation(libs.material)
    implementation(libs.activity)
    implementation(libs.constraintlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.ext.junit)
    androidTestImplementation(libs.espresso.core)
    implementation(libs.firebase.firestore)
    implementation(platform("com.google.firebase:firebase-bom:34.2.0"))
    implementation("com.google.firebase:firebase-analytics")
}
```

Código de activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <Button
        android:id="@+id/buttonCargar"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="48dp"
        android:layout_marginTop="272dp"
        android:text="Leer Colección de FireStore"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <ProgressBar
        android:id="@+id/progressBar"
        style="?android:attr/progressBarStyle"
        android:layout_width="106dp"
        android:layout_height="84dp"
        android:layout_marginStart="120dp"
        android:layout_marginTop="120dp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonCargar"
        tools:visibility="visible" />


    <TextView
        android:id="@+id/tvDocuments"
        android:layout_width="321dp"
        android:layout_height="345dp"
        android:layout_marginStart="48dp"
        android:layout_marginTop="24dp"
        android:textAlignment="gravity"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/buttonCargar" />

    <EditText
        android:id="@+id/edtNombres"
        android:layout_width="258dp"
        android:layout_height="46dp"
        android:layout_marginStart="41dp"
        android:layout_marginTop="37dp"
        android:ems="10"
        android:hint="Ingrese nombres..."
        android:inputType="text"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

```
    <EditText
        android:id="@+id/edtApellidos"
        android:layout_width="262dp"
        android:layout_height="55dp"
        android:layout_marginStart="41dp"
        android:layout_marginTop="15dp"
        android:ems="10"
        android:hint="Ingrese apellidos..."
        android:inputType="text"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/edtNombres" />


<com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabAddDoc"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="98dp"
        android:layout_marginEnd="36dp"
        android:clickable="true"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/edtApellidos"
        app:layout_constraintTop_toTopOf="parent"
        app:srcCompat="@android:drawable/ic_menu_add" />

    <EditText
        android:id="@+id/edtIdDoc"
        android:layout_width="251dp"
        android:layout_height="55dp"
        android:layout_marginStart="48dp"
        android:layout_marginBottom="8dp"
        android:ems="10"
        android:hint="Ingrese ID para buscar..."
        android:inputType="text"
        app:layout_constraintBottom_toTopOf="@+id/buttonCargar"
        app:layout_constraintStart_toStartOf="parent" />


<com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabDelDol"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="18dp"
        android:layout_marginTop="58dp"
        android:layout_marginEnd="38dp"
        android:clickable="true"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@+id/edtIdDoc"
        app:layout_constraintTop_toBottomOf="@+id/fabAddDoc"
        app:srcCompat="@android:drawable/ic_menu_delete" />


</androidx.constraintlayout.widget.ConstraintLayout>
```

Código de libs.versions.toml

```
[versions]
agp = "8.13.0"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
appcompat = "1.7.1"
material = "1.13.0"
activity = "1.11.0"
constraintlayout = "2.2.1"
googleGmsGoogleServices = "4.4.3"
firebaseFirestore = "26.0.0"


[libraries]
junit = { group = "junit", name = "junit", version.ref = "junit" }
ext-junit = { group = "androidx.test.ext", name = "junit", version.ref =
"junitVersion" }
espresso-core = { group = "androidx.test.espresso", name = "espresso-
core", version.ref = "espressoCore" }
appcompat = { group = "androidx.appcompat", name = "appcompat",
version.ref = "appcompat" }
material = { group = "com.google.android.material", name = "material",
version.ref = "material" }
activity = { group = "androidx.activity", name = "activity", version.ref
= "activity" }
constraintlayout = { group = "androidx.constraintlayout", name =
"constraintlayout", version.ref = "constraintlayout" }
firebase-firestore = { group = "com.google.firebase", name = "firebase-
firestore", version.ref = "firebaseFirestore" }

[plugins]
android-application = { id = "com.android.application", version.ref =
"agp" }
google-gms-google-services = { id = "com.google.gms.google-services",
version.ref = "googleGmsGoogleServices" }
```

Código de MainActivity.java

```
package sv.edu.ues.dam235.appdemombas;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ProgressBar;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import
com.google.android.material.floatingactionbutton.FloatingActionButton;
import com.google.firebase.firestore.DocumentSnapshot;
import com.google.firebase.firestore.FirebaseFirestore;
import com.google.firebase.firestore.QueryDocumentSnapshot;
import com.google.firebase.firestore.QuerySnapshot;

import org.checkerframework.checker.units.qual.N;

public class MainActivity extends AppCompatActivity {

    Button btCargar;
    TextView tvJSON;
    ProgressBar progressBar;
    FloatingActionButton fabAddDoc, fabDelDoc;
    EditText edtNombres, edtApellidos,edtID;
    FirebaseFirestore db;


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v,
insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top,
systemBars.right, systemBars.bottom);
            return insets;
```

```
        });

        db = FirebaseFirestore.getInstance();

        btCargar = findViewById(R.id.buttonCargar);
        tvJSON  = findViewById(R.id.tvDocuments);
        progressBar = findViewById(R.id.progressBar);
        progressBar.setVisibility(View.GONE);
        edtNombres = findViewById(R.id.edtNombres);
        edtApellidos = findViewById(R.id.edtApellidos);
        edtApellidos = findViewById(R.id.edtApellidos);
        edtID = findViewById(R.id.edtIdDoc);
        fabAddDoc = findViewById(R.id.fabAddDoc);
        fabDelDoc = findViewById(R.id.fabDelDol);

        btCargar.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if(edtID.getText().toString().isEmpty()){
                    actualizarListaDocumentos(view);
                }else{

                    String idDoc = edtID.getText().toString();

db.collection("alumnos").document(idDoc).get().addOnCompleteListener(new
OnCompleteListener<DocumentSnapshot>() {
                        @Override
                        public void onComplete(@NonNull
Task<DocumentSnapshot> task) {
                            if(task.isSuccessful()){
                                DocumentSnapshot document =
task.getResult();
                                if(document.exists()){
                                    StringBuilder documento = new
StringBuilder();
                                    if(document.getData()!=null){
                                        documento.append("ID de
documento: \n")
                                                .append(document.getId())
                                                .append("\nCampos:")
                                                .append("\n{")

.append("\n").append(document.getData())
                                                .append("\n}\n\n");
                                        tvJSON.setText(documento);
                                    }
                                }else{
                                    Toast.makeText(MainActivity.this, "No
existe el documento", Toast.LENGTH_SHORT).show();
                                }
                            }else{
                                Toast.makeText(MainActivity.this, "Error
al obtener el documento", Toast.LENGTH_SHORT).show();
                            }
                        }
                    });
                }
```

```
            }
        });

        fabAddDoc.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View view) {

                if(!edtApellidos.getText().toString().isEmpty() &&
!edtNombres.getText().toString().isEmpty() &&
!edtID.getText().toString().isEmpty()){
                    actualizarDocumento(edtID.getText().toString(),
view);
                    return;
                }

                String nombres = edtNombres.getText().toString();
                String apellidos = edtApellidos.getText().toString();
                if (!nombres.isEmpty() && !apellidos.isEmpty()) {
                    db.collection("alumnos").add(new Alumno(nombres,
apellidos).toMap());
                    Toast.makeText(MainActivity.this, "Documento creado",
Toast.LENGTH_SHORT).show();
                    actualizarListaDocumentos(view);
                    edtNombres.setText("");
                    edtApellidos.setText("");
                } else {
                    Toast.makeText(MainActivity.this, "Ingrese nombres y
apellidos", Toast.LENGTH_SHORT).show();
                }
            }
        });

        fabDelDoc.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                String idDoc = edtID.getText().toString();
                if (!idDoc.isEmpty()) {
                    db.collection("alumnos").document(idDoc).delete();
                    Toast.makeText(MainActivity.this, "Documento
eliminado", Toast.LENGTH_SHORT).show();
                    edtID.setText("");
                    actualizarListaDocumentos(view);
                } else {
                    Toast.makeText(MainActivity.this, "Ingrese el ID del
documento a eliminar", Toast.LENGTH_SHORT).show();
                }
            }
        });


    }

    private void actualizarListaDocumentos(View view) {
        progressBar.setVisibility(View.VISIBLE);
        tvJSON.setText("");
        progressBar.setVisibility(View.VISIBLE);
        StringBuilder documento = new StringBuilder();
```

```
        db.collection("alumnos")
                .get()
                .addOnCompleteListener(new
OnCompleteListener<QuerySnapshot>() {
                    @Override
                    public void onComplete(@NonNull Task<QuerySnapshot>
task) {
                        progressBar.setVisibility(View.GONE);
                        if (task.isSuccessful()) {
                            if (task.getResult().size() == 0) {
                                tvJSON.setText("No data to show...");
                            } else {
                                for (QueryDocumentSnapshot document :
task.getResult()) {
                                    documento.append("ID de documento:
\n")
                                            .append(document.getId())
                                            .append("\nCampos:")
                                            .append("\n{")

.append("\n").append(document.getData())
                                            .append("\n}\n\n");
                                    tvJSON.setText(documento);
                                }
                            }
                        } else {
                            tvJSON.setText("Error al obtener
documentos.");
                        }
                    }
                });

    }

    private void actualizarDocumento(String idDoc, View view) {


db.collection("alumnos").document(idDoc).get().addOnCompleteListener(new
OnCompleteListener<DocumentSnapshot>() {
            @Override
            public void onComplete(@NonNull Task<DocumentSnapshot> task)
{
                if (task.isSuccessful()) {
                    DocumentSnapshot document = task.getResult();
                    if (document.exists()) {

                        if (document.getData() != null) {

db.collection("alumnos").document(idDoc).update("nombres",
edtNombres.getText().toString(),
                                    "apellidos",
edtApellidos.getText().toString());
                            Toast.makeText(MainActivity.this, "Documento
actualizado", Toast.LENGTH_SHORT).show();
                            actualizarListaDocumentos(view);
                            edtNombres.setText("");
                            edtApellidos.setText("");
```

```
                            edtID.setText("");
                        }
                    } else {
                        Toast.makeText(MainActivity.this, "No existe el
documento a actualizar", Toast.LENGTH_SHORT).show();
                    }
                }
            }

        });

    }
}
```